ACCEPT: Adaptive Codebook for Composite and Efficient Prompt Tuning

Anonymous ACL submission

Abstract

Prompt Tuning has been a popular Parameter-Efficient Fine-Tuning method attributed to its remarkable performance with few updated parameters on various large-scale pretrained Language Models (PLMs). Traditionally, each prompt has been considered indivisible and updated independently, leading to a linear increase in parameters as prompt length grows. To address this issue, we propose **A**daptive **C**odebook for **C**omposite and **E**fficient **P**rompt **T**uning (**ACCEPT**). In our method, we refer to the concept of product quantization (PQ), allowing all soft prompts to share a set of learnable codebook vectors in each subspace, with each prompt differentiated by a set of adaptive weights. We achieve the superior performance on 17 diverse natural language tasks including natural language understanding (NLU) and question answering (QA) tasks by tuning only 0.3% of parameters of the PLMs. Our approach also excels in few-shot and large model settings, highlighting its significant potential.

1 Introduction

With the blooming of large language models, Parameter Efficient Fine-Tuning becomes an effective solution to leverage the power of pretrained language models (LMs). Among various approaches, Prompt Tuning (PT) has been recognized for its simplicity and efficacy by adding tokens in front of the inputs. Though prompting pretrained LMs with specific or human-designed instructions makes model transferable to downstream tasks, additional effort is needed for elaborating the prompts as the output produced are often sensitive to them. To address this issue, learning the prompts becomes a solution. Prompt tuning (Lester et al., 2021), Prefix tuning (Li and Liang, 2021) and P-tuning (Liu et al., 2022) replace explicit instructions with continuous prompt embeddings and provide flexibilities for the pretrained models to adapt themselves with superior performance. Following the concept, ATTEMPT (Asai et al., 2022), MPT (Wang et al., 2023), DePT (Shi and Lipani, 2024), and TPT (Wu et al., 2023) demonstrate the capability of learnable PT in both single and multitask training scenarios.

However, previous studies often treat the prompts as independent units in learning. Though the learned prompts can be further clustered for noise filtering (Bhardwaj et al., 2022), the parameters needed for training are not reduced since learning occurs before clustering. In this work, we introduce a method that represents the prompt based on a set of *learnable codewords*. All prompts share a codebook with $\mathcal{N}$ codewords. Compared with updating the prompts independently and thus preventing word embeddings from sharing information with each other, our codebooks are sharable in a downstream task and avoid causing the number of parameters to grow linearly with the prompts.

In addition, our approach does not follow the common practice of regarding each prompt as inseparable. When treating a prompt as an indivisible word embedding, we may overlook the possibility that, say, certain words may align with other words in the first half of the embedding and match different words in the second half. To tackle this issue, we adopt the idea of product quantization (PQ) (Jegou et al., 2010) by dividing a prompt's word embedding into several subsections and construct a codebook for each subsection. In the past, PQ is effective for approximate nearest-neighbor search (Jegou et al., 2010; Yu et al., 2018) and neural network compression (Wu et al., 2016). However, if we directly apply PQ to the learned parameters, their amount will not be lowered for training. Hence, we simply follow PQ's concept where the codebooks are subsection-specific, and provide a set of learnable codewords for each subsection. This makes prompts share some identical subvectors, which allows part of tokens to have the same characteristics in a more fine-grained dimension.

To ease the learning process and make it dif-

ferentiable, we allow each subvector to be softly combined (via linear coefficients) with the codewords, rather than being assigned by only one of the codewords as in PQ. This increases both diversity and flexibility of the representation. In the past, learning-based PT can be typically done by increasing the input length with prepended prompts (Lester et al., 2021; Asai et al., 2022; Wu et al., 2023), or further adding the original embedding of words with the same number of additional prompts (Shi and Lipani, 2024). Our method, referred to as **ACCEPT**, is generally applicable and works for both. We conduct experiments on 17 natural language tasks and show that our method consistently outperforms previous PT approaches.

2 Related Work

Parameter-efficient fine-tuning enhances the capabilities of pretrained LMs by updating a small set of parameters. The approach varies, such as training extra modules (Houlsby et al., 2019; Sung et al., 2022) or modifying specific parts like biases or attention weights (Zaken et al., 2022; Hu et al., 2021). Among these, as Prompt Tuning (PT) is popular for its simplicity and effectiveness, we focus on PT.

2.1 Prompt Tuning Methods

This track focuses on enhancing the quality and efficiency of prompts. Schick and Schütze (2021) and Brown et al. (2020) incorporate manually crafted instructions into the input sequence to provide task-specific guidance helping steer the model's output. When the instructions are well-designed, models with frozen parameters exhibit excellent performance. However, additional effort is required for human adjustment since the output is sensitive to the prompts. To address the issue, Shin et al. (2020), Wang et al. (2021) and Gao et al. (2021) further generate hard prompt templates by model automatically. Nonetheless, optimizing discrete prompts is challenging. Thus, Prompt Tuning (Lester et al., 2021), Prefix tuning (Li and Liang, 2021), and P-tuning (Liu et al., 2022) turn prompts into continuous vectors, known as soft prompt, which are prepended to the word embeddings. The learnable prompts are trained with the pretrained LMs frozen. By turning discrete prompts to a continuous space, the optimization can be achieved by a simple gradient descent. Recently, Su et al. (2021) and Spot (Vu et al., 2022) explore the advantages of initializing prompts by pretrained ones from other tasks. They demonstrate that learning prompts on one or more source tasks, and subsequently utilizing these learned prompts as initializations for a target task, is notably effective. ATTEMPT (Asai et al., 2022), MPT (Wang et al., 2023) and TPT (Wu et al., 2023) further design various architectures for multitask transfer learning. On the other hand, DePT (Shi and Lipani, 2024) focuses on reducing the training and inference time by decomposing prompt as a shorter one and a low-rank matrix added on word embeddings. Nevertheless, earlier approaches treat prompt as monolithic units, causing the number of trainable parameters to increase proportionally to the prompt length. In contrast, our method introduces a shared codebook in each subspace, which remains unaffected by the prompt length and facilitates information sharing among different prompts.

2.2 Quantization in NLP

Vector quantization (VQ) is a related technique to PQ which is widely employed in NLP. VQ provides an effective discretization of latent sentence representations, making it especially suitable for NLP tasks due to the inherently discrete nature of text, as demonstrated in Van Den Oord et al. (2017), Roy et al. (2018), Roy and Grangier (2019), Mercatali and Freitas (2021) and Angelidis et al. (2021). VQ is also used in PT. Bhardwaj et al. (2022) initially train a contextualized prompt for each input and cluster them using VQ to reduce variance.

However, in the previous approaches, the number of parameters remains substantial since the training of original representations occurs before clustering. Different from these methods, we introduce learnable codebooks and adaptive weights which enable end-to-end training, thereby maintaining parameter efficiency throughout the process.

3 Methodology

We first give a preliminary of PT for downstream tasks and PQ, and then present our method.

3.1 Prompt Tuning for Downstream Tasks

Given a pretrained LM with parameters θ, we want to transfer it to a target task with the training data $D = \{(x_j, y_j)\}|_{j=1}^{|D|}$. We first map x_j to a word embedding e_j as input, where $e_j \in R^{l \times d}$, l is the maximal input sequence length and d is the word-embedding dimension. PT prepends a set of trainable continuous prompt embeddings

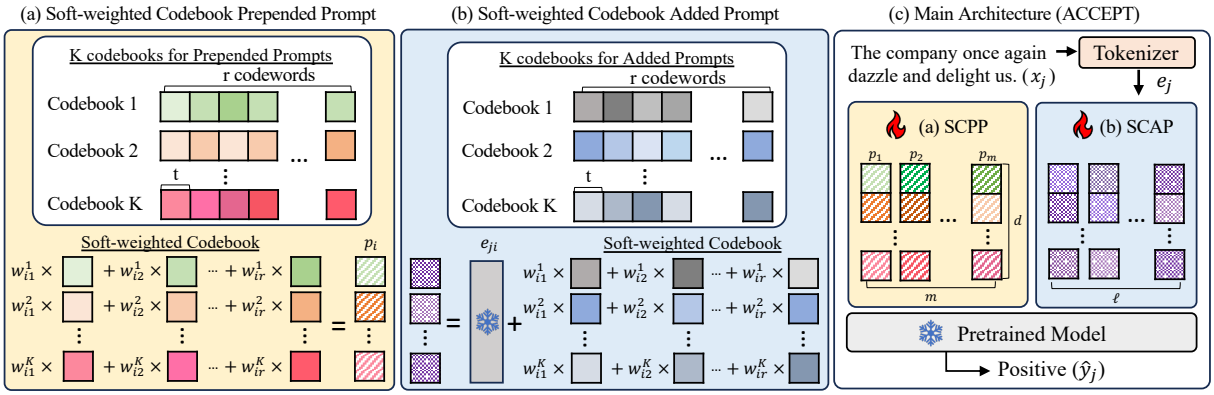


Figure 1: The overall model architecture of **ACCEPT**. We subdivide both (a) *S*oft-weighted *C*odebook *P*repended *P*rompt (*SCPP*) and (b) *S*oft-weighted *C*odebook *A*dded *P*rompt (*SCAP*) to K subspaces. Each subspace has a codebook with r codewords shared by all prompts. Each sub-prompt is linearly combined by the codewords and weights. (c) In the main architecture of ACCEPT, the final input is formed by prepending *SCPP* to the word embedding updated with *SCAP*. The pretrained model, with its parameters fixed, learns to output correct labels through tunable *SCPP* and *SCAP*.

$P = \{p_1, p_2, ...p_m\}$ ($p_i \in R^d$) to the input embeddings while keeping the pretrained model parameters θ fixed. The training goal is to maximize the output probability of target y_j as below,

$$\max_{P} \sum_{j=1}^{|D|} p_\theta(y_j|\theta; [P, e_j]). \quad (1)$$

3.2 Review of PQ and Method Motivation

VQ is known as the process of mapping a vector x to the closest codeword c^* in a codebook $C = \{c_1, c_2, ..., c_N\}$ containing N codewords. As an extension, PQ divides the vector $x \in R^d$ into K subspace, $x = [x^1, x^2, ..., x^K]$, with $d = tK$ and $x^k \in R^t$. Each subspace possesses a codebook C^k which contains N_k codewords of dimension t for $k = 1 \cdots K$. PQ thus exploits the Cartesian product of the codeword sets,

$$C = C^1 \times C^2 \times \cdots \times C^K, \quad (2)$$

to encode the vector x. The total number of codewords becomes $\mathcal{N} = \Pi_{k=1}^{K} N_k$ for entire space. When $K = 1$, PQ degenerates to VQ.

PQ has the advantage of enabling more codewords for the representation of x by consuming fewer parameters. Eg., if N_k is the same for all k, PQ can take the storage cost of only $O(tKN_k) = O(dN_k)$ to provide the codewords amount of $(N_k)^K$. For VQ, however, only N_k codewords are provided under the same storage cost, or the storage should be increased to $O(d(N_k)^K)$ to get the same amount of codewords. Hence, PQ is more parameter-efficient and suitable for PT. The codewords distributed in subsections can enrich the diversity and flexibility of representation for the sub-problem solving.

However, as mentioned above, parameter-efficient learning is not attainable if we perform PT first and then PQ. Hence, our method does not really do the 'quantization' step but only takes PQ's idea of efficient representation and makes the codewords of all subspaces learnable for PT. Moreover, for each subspace, we do not use only one codeword to express the input x for that subsection, but softly combining the codewords with linear coefficients for a more precise representation. Details are given below.

3.3 Proposed Method – ACCEPT

Previous methods often view each prompt as a single and indivisible word embedding, independent to other prompts. We suppose that tokens can share the same characteristics in a more fine-grained dimension. Our method leverages the concept of PQ and partition embedding space into K smaller subspaces. The k-th subspace has a codebook $\mathcal{C}^k = \{c_1^k, c_2^k, ..., c_r^k\}$ containing r codewords of dimension t, with $t = d/K$. Specifically, the total K codebooks are shared across all prompts.

Remember that there is a set of trainable prompts $P = \{p_1, p_2, ..., p_m\}$ ($p_i \in R^d$) for a downstream task in PT. Similarly, we divide each p_i into K sub-prompts $p_i = \{p_i^1, p_i^2, ..., p_i^K\}$ ($p_i \in R^t$). We assign a group of weights, $W_i = \{w_i^1, w_i^2, ..., w_i^K\}$ ($w_i^k \in R^r$), to the i-th prompt in every subspace. A

sub-prompt p_i^k is then expressed as a linear combination of the codewords in $\mathcal{C}^k$ using the coefficient weights $w_i^k = \{w_{i1}^k, w_{i2}^k, ..., w_{ir}^k\}$. Thus, the k-th subvector of the i-th prompt is calculated as

$$p_i^k = c_1^k \times w_{i1}^k + c_2^1 \times w_{i2}^k + ... + c_r^k \times w_{ir}^k, \quad (3)$$

where c_j^k is the j-th codeword ($j = 1, \cdots, r$) in the k-th codebook and w_{ij}^k is the weight for c_j^k in the i-th prompt, respectively.

The prompts in P thus depend on both the sets of codewords, $\mathbf{C} = \{\mathcal{C}^k | k = 1, \cdots, K\}$ and combination weights, $\mathbf{W} = \{W_i | i = 1, \cdots, m\}$, in our method. To reflect this, we denote it as $P(\mathbf{C}, \mathbf{W})$ in the following. Our goal is to maximize the output probabilities of the ground truth label y_j as

$$\max_{\mathbf{C},\mathbf{W}} \sum_{j=1}^{|D|} p_\theta(y_j|\theta; [P(\mathbf{C}, \mathbf{W}), e_j]), \quad (4)$$

where $P(\mathbf{C}, \mathbf{W}) \in R^{m \times d}$. We refer Eq. 4 to as the Soft-Weighted codebook Prepended Prompt (*SCPP*) tuning in our method, as the prompts are prepended to the inputs. Fig. 1(a) gives an illustration of *SCPP* learning.

As mentioned in Sec. 1, PT can also be done by adding complementary prompts to the original embedding (Shi and Lipani, 2024), and our method works for both. To achieve this, we conduct another prompt set Q that contains the same number of (l) prompts of the same length (d) to the input word embedding e_j ($e_j \in R^{l \times d}$). Similar mechanisms are applied to Q, which depends on the learnable codewords $\mathbf{C}'$ and combination weights $\mathbf{W}'$ too. We optimize $\mathbf{C}'$ and $\mathbf{W}'$ by solving

$$\max_{\mathbf{C}',\mathbf{W}'} \sum_{j=1}^{|D|} p_\theta(y_j|\theta; [e_j + Q(\mathbf{C}', \mathbf{W}')]), \quad (5)$$

where $Q(\mathbf{C}', \mathbf{W}') \in R^{l \times d}$ having the same shape of e_j. We refer Eq. 5 to as the Soft-Weighted codebook Added Prompt (*SCAP*) tuning, as the prompts are added to the original word embeddings as updates. Fig. 1(b) illustrates the *SCAP* learning. Combing *SCPP* and *SCAP* then forms our final **ACCEPT** (shown in Fig. 1(c)). At the same scale of parameters, combining the two types of prompts reduces the total input length, which makes training and inference more efficient (Shi and Lipani, 2024). **ACCEPT** then learns by maximizing

$$\max_{\mathbf{C},\mathbf{W},\mathbf{C}',\mathbf{W}'} \sum_{j=1}^{|D|} p_\theta(y_j|\theta; [P(\mathbf{C}, \mathbf{W}), e_j + Q(\mathbf{C}', \mathbf{W}')]). \quad (6)$$

With only $[\mathbf{C}, \mathbf{W}]$, $[\mathbf{C}', \mathbf{W}']$ trainable and θ frozen.

Number of Parameters. With the vanilla PT, for a model having embedding dimension d and m prompts, the number of parameters is md. As we subdivide the embedding into K subspaces, each t-dimensional ($t = d/K$) subspace has r codewords, and thus the number of parameters of each codebook is rt. Total K codebooks then need rtK parameters. As for the weights, each prompt has r weights in K subspaces, which contains total rK parameters. Multiplied by the number of prompts m forms a total of rmK parameters. Finally, the total parameters of our method is as below,

$$\underbrace{rtK}_{\#para.of codebook} + \underbrace{rmK}_{\#para.of weight} = rd + rmK. \quad (7)$$

Note that the number of parameters for the codebook is independent of the number of prompts, preventing a proportional growth with m. To maintain the same scale of the trainable parameters with vanilla PT for a better comparison, we set r by letting $rd + rmK \leq md$. This ensures the number of parameter usage is no greater than the vanilla PT.

4 Experiments

We present the experimental results and comparisons to other approaches in this section.

4.1 Experimental Settings

Datasets and Tasks. Following previous works, we evaluate our method on 13 NLU tasks and 4 QA tasks, including (1) MNLI (Williams et al., 2018), QQP, QNLI (Demszky et al., 2018), SST-2 (Socher et al., 2013), STS-B (Cer et al., 2017), MRPC (Dolan and Brockett, 2005), RTE (Giampiccolo et al., 2007) and CoLA (Warstadt et al., 2019) from GLUE (Wang et al., 2018) benchmark; (2) MultiRC (Khashabi et al., 2018), BoolQ (Clark et al., 2019), WiC (Pilehvar and Camacho-Collados, 2018), WSC (Levesque et al., 2012) and CB (De Marneffe et al., 2019) from SuperGLUE (Wang et al., 2019) benchmark; (3) MRQA 2019 Shared Task (Fisch et al., 2019), including Natural Questions (Kwiatkowski et al., 2019), HotpotQA (Yang et al., 2018), SearchQA (Dunn et al., 2017) and NewsQA (Trischler et al., 2016).

Baselines. We compare the proposed approach with various PEFT baselines including: (1) Fully fine-tuning (FT), where all the parameters of the pretrained backbone models are updated; (2) Prompt Tuning (PT), where prompts are initialized

by randomly sampled top vocabularies; (3) Some variants of PT, changing prompt architectures or utilizing knowledge transfer from other tasks such as SPoT (Vu et al., 2022), ATTEMPT (Asai et al., 2022), MPT (Wang et al., 2023), TPT (Wu et al., 2023) and DePT (Shi and Lipani, 2024); (4) Other PEFT methods including Adapter (Houlsby et al., 2019) and AdapterDrop (Rücklé et al., 2021), inserting lightweight modules in the middle blocks of the pretrained models; BitFit (Zaken et al., 2022), updating the bias terms in the attention mechanism; LoRA (Hu et al., 2021), updating the attention weights with two additional low-rank matrices; LST (Sung et al., 2022), transferring by a ladder-side network-based adapter; Hyperformer (Mahabadi et al., 2021) and HyperDecoder (Ivison and Peters, 2022), training a module to output the weights of adapters.

Models. To provide a fair comparison with the previous methods, the main experiments are performed on the T5-base model with 220M parameters and $d = 768$. We also conduct experiments on other models with various scales including T5-small, T5-large, T5-3B and Flan-T5-11B models with 60M, 770M, 3B, and 11B parameters, respectively. The model dimensions are 512, 1024, 1024 and 1024, respectively. Note that Flan-T5-11B is an enhanced version of T5 model that has been fine-tuned in a collection of tasks.

Implementation Details. In the main experiments on GLUE, SuperGLUE and MRQA datasets, we divide the embedding into $K = 24$ and $K = 2$ subsections for *SCPP* and *SCAP*, respectively, where the parameters are chosen based on the performances on a small dataset RTE (detailed in Sec. 4.4). We set the associated number of codewords r to maintain the total number of parameters no bigger than PT with $m = 100$. We conduct three initialization strategies for the codebooks and weights of *SCPP* and *SCAP* in ACCEPT: (1) randomly initialized, (2) initialized by the pretrained weights of the intermediate tasks (we use MNLI for GLUE/SuperGLUE task and SQuAD (Rajpurkar et al., 2016) for QA tasks following Vu et al. (2022)), and (3) initialized by the target task itself. For the latter two strategies, we first train *SCPP* and *SCAP* and use the pretrained weights as initialization for ACCEPT. On each dataset, we select the best strategy as the final results following Wu et al. (2023). In the few-shot experiments, following Mahabadi et al. (2021) and Asai et al. (2022), we sample $\gamma = \{4, 16, 32\}$ training instances three times

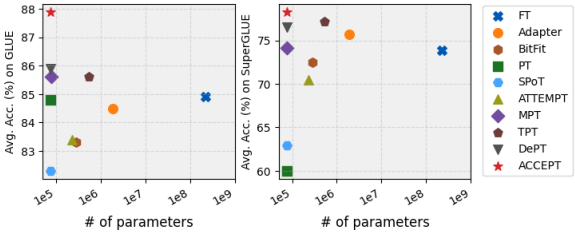


Figure 2: Average performance on the GLUE and SuperGLUE benchmarks relative to the number of trainable parameters for the T5-base model. ACCEPT achieves the best performance with the fewest parameters.

with different seeds and report the average and standard deviation of our results. *SCPP* and *SCAP* are pretrained with one of the selected source dataset (MNLI, QQP, SST-2, SQUAD (Rajpurkar et al., 2016), and ReCoRD (Zhang et al., 2018)) following the previous methods (Su et al., 2021), (Asai et al., 2022), (Shi and Lipani, 2024). More detailed experiment setup is listed in Appendix A.

4.2 Results on NLU and QA Tasks

In Tab. 1, we compare the performances and the number of parameters during training of the proposed method with various methods on GLUE and SuperGLUE benchmarks. As can be seen, our method outperforms previous PT methods by a large margin, especially on MRPC, RTE and CoLA datasets of the GLUE benchmark, while consistently improving on other datasets such as MNLI, QQP, etc. Similar results can be found on the SuperGLUE benchmark. Our method achieves a great improvement on the Bool, WiC and CB datasets, while also yielding promising performances on MultiRC and WSC. It is worth noting that our method surpasses previous PEFT methods exploiting much more tunable parameters such as Adapter, and also outperforms FT by 3.0% and 4.4% on the average performances of GLUE and SuperGLUE with only 0.3% parameters tuned. We further visualize the average performances against the number of trainable parameters for each method in Fig. 2. Our approach achieves the highest average accuracy while using the fewest parameters, making it more suitable for both performance and parameter efficiency than the others.

Besides having favorable results on the NLU tasks above, the proposed method also achieves nice performances on QA tasks. Tab. 2 demonstrates that our method achieves a 4.2% improvement on the average of MRQA 2019 Shared Task

Method	#Para.	GLUE									SuperGLUE					
		MNLI	QQP	QNLI	SST-2	STS-B	MRPC	RTE	CoLA	Avg.	Multi	Bool	WiC	WSC	CB	Avg.
Fine-tuning	220M	**86.8**	**91.6**	93.0	94.6	89.7	90.2	71.9	61.8	84.9	72.8	81.1	70.2	59.6	85.7	73.9
Adapter	1.9M	86.5	90.2	93.2	93.8	90.7	85.3	71.9	64.0	84.5	**75.9**	**82.5**	67.1	**67.3**	85.7	75.7
AdapterDrop	1.1M	86.3	90.2	93.2	93.6	**91.4**	86.3	71.2	62.7	84.4	72.9	82.3	68.3	**67.3**	85.7	75.3
BitFit	280K	85.3	90.1	93.0	94.2	90.9	86.8	67.6	58.2	83.3	74.5	79.6	70.0	59.6	78.6	72.5
LoRA	3.8M	86.3	89.0	93.2	94.3	90.9	90.1	75.5	63.3	85.3	72.6	81.3	68.3	**67.3**	92.9	76.5
LST	3.8M	85.6	88.8	93.3	94.1	90.7	90.4	71.9	58.1	84.1	–	–	–	–	–	–
HyperFormer(m)	638K	85.7	90.0	93.0	94.0	89.7	87.2	75.4	63.7	84.8	72.9	**82.5**	69.0	**67.3**	85.7	75.4
HyperDecoder(m)	1.8M	86.0	90.5	**93.4**	94.0	90.5	87.7	71.7	55.9	83.7	70.4	78.8	67.1	61.5	82.1	72.0
PT	76.8K	81.3	89.7	92.8	90.9	89.5	68.1	54.7	10.6	72.2	58.7	61.7	48.9	51.9	67.9	57.8
PT†	76.8K	83.4	90.2	93.1	91.9	90.2	90.1	78.8	60.7	84.8	65.7	63.7	50.8	51.9	67.9	60.0
SPoT	76.8K	85.4	90.1	93.0	93.4	90.0	79.7	69.8	57.1	82.3	74.0	77.2	67.0	50.0	46.4	62.9
ATTEMPT	232K	84.3	90.3	93.0	93.2	89.7	85.7	73.4	57.4	83.4	74.4	78.8	66.8	53.8	78.6	70.5
MPT	77.6K	85.9	90.3	93.1	93.8	90.4	89.1	79.4	62.4	85.6	74.8	79.6	69.0	**67.3**	79.8	74.1
DePT	76.8K	85.0	90.4	93.2	94.2	90.8	90.7	79.1	63.8	85.9	74.3	79.3	68.7	**67.3**	92.9	76.5
TPT[1]	539K	85.5	90.1	93.2	**94.7**	89.8	89.7	82.3	59.8	85.6	74.4	80.1	69.8	**67.3**	94.6	77.2
ACCEPT (Ours)	74.9K	85.9	90.4	93.3	94.5	91.0	**93.1**	**86.3**	**68.8**	**87.9**	74.9	82.3	**70.5**	**67.3**	**96.4**	**78.3**

Table 1: Performance on GLUE and SuperGLUE with T5-base model. For comparisons with prior works, we use Pearson Correlation for STS-B, Matthews Correlation for CoLA, F1 for MultiRC (Multi), and accuracy for other tasks as metrics. [1] sourced from Wu et al. (2023) and the others sourced from Shi and Lipani (2024). †The values are the improved results tuned by Shi and Lipani (2024).(m) refers to multi-task training.

Method	#Para.	MRQA				
		NQ	HP	SQA	News	Avg.
Fine-tuning	220M	75.1	77.5	81.1	65.2	74.7
Adapter	1.9M	74.2	77.6	81.4	65.6	74.7
BitFit	280K	70.7	75.5	77.7	64.1	72.0
LoRA	3.8M	72.4	62.3	72.5	56.9	66.0
PT	76.8K	67.9	72.9	75.7	61.1	69.4
SPoT	76.8K	68.2	74.8	75.3	58.2	69.1
ATTEMPT	232K	70.4	75.2	77.3	62.8	71.4
MPT	77.6K	$72.0_{0.1}$	$75.8_{0.1}$	$77.2_{0.1}$	$63.7_{0.1}$	72.2
DEPT	76.8K	$73.2_{0.1}$	$76.8_{0.3}$	$77.6_{0.2}$	$64.4_{0.1}$	73.0
ACCEPT (Ours)	74.2K	$73.6_{0.05}$	$77.1_{0.09}$	$78.9_{0.01}$	$64.6_{0.06}$	73.6

Table 2: Performance on the MRQA 2019 Shared Task. We report the average F1 score and standard deviation of three experiments with different seeds. The proposed method achieves promising performances with the limited number of parameters.

than PT with fewer parameters, further reducing the performance gap between FT and PT methods.

To conclude, the proposed method achieves state-of-the-art performances on the challenging GLUE/SuperGLUE benchmarks and MRQA 2019 Shared Task with fewer trainable parameters, highlighting its efficiency and effectiveness.

4.3 Results on Few-shot Adaptation

Following Gao et al. (2021), Asai et al. (2022), Wang et al. (2023), Wu et al. (2023), Shi and Lipani (2024), we conduct the experiments with a limited number of training samples available on the BoolQ, MultiRC and WiC datasets to verify the capability of ACCEPT in resource-limited scenarios. The experimental process involves initially training prompts on the intermediate tasks (e.g., MNLI) followed by transferring them to the target datasets with 4, 16, or 32 randomly sampled instances. In Tab. 3, our method accomplishes impressive results on BoolQ dataset, which is consistent with Tab. 1. It also outperforms the previous methods on CB dataset with 4 shots. Note that for the CB dataset with 16 and 32 shots, our approach outperforms most of the methods except for ATTEMPT and TPT both using much more parameters than ours. The results demonstrate that ACCEPT remains effective in the few-shot adaptation scenarios.

4.4 Ablation Study

Learnable Codebook and Subdivision. To demonstrate the effectiveness of ACCEPT, we first conduct an ablation study of PQ, utilizing the shared learnable codebook and prompt embedding subdivision, with the prepended and added prompt tunings. Tab. 4 shows that by sharing the learnable codebook among prompts, there is a noticeable performance improvement over the original architectures. Moreover, by dividing prompt embeddings into more fine-grained pieces, the performances are further enhanced. The results reveal the efficacy of PQ by subdividing the prompt embedding space.

Different Granularity of Subdivision. We further study on the impact of using different sub-dimension (t) and codebook size (r) pairs in our approach. We choose multiple values for K and divide the embeddings of dimension d into multiple vectors with sub-dimension t. We then determine r to satisfy $rtK + rmK \leq 100d$, ensuring fewer pa-

Task	γ-shot # Para.	FT 220M	AD 1.9M	PT 76.8K	ST 76.8K	HF 638K	(IA)3 55.3K	ATP 232K	MPT 77.6K	TPT 538K	DEPT 76.8K	ACCEPT (Ours) 74.9K
BoolQ	4	50.5	53.4	61.6	50.5	48.0	56.7	61.8	62.2	62.2	$62.7_{5.4}$	$70.5_{1.6}$
	16	56.5	51.4	61.9	50.6	50.2	62.0	60.0	63.3	63.5	$66.9_{4.4}$	$71.9_{1.3}$
	32	58.4	54.5	61.7	61.2	58.3	67.2	65.3	68.9	67.4	$67.2_{3.4}$	$72.5_{1.0}$
CB	4	57.7	51.1	53.5	71.4	60.7	65.5	82.1	73.6	78.6	$75.0_{5.1}$	$78.6_{3.6}$
	16	77.0	74.8	63.5	64.3	76.3	71.4	78.5	78.6	80.4	$78.6_{4.3}$	$81.0_{2.0}$
	32	80.0	74.8	67.8	64.3	81.4	75.0	85.7	82.1	86.3	$82.1_{2.3}$	$83.3_{2.0}$
SciTail	4	79.6	79.5	57.7	69.6	82.0	65.4	80.2	80.2	81.0	$78.1_{2.5}$	$79.0_{4.4}$
	16	80.0	83.2	60.8	71.9	86.5	74.4	79.5	87.3	85.5	$78.5_{1.4}$	$80.5_{3.1}$
	32	81.9	85.0	60.2	71.9	85.8	80.4	80.2	86.3	85.2	$85.4_{3.1}$	$84.8_{0.4}$

Table 3: Few-shot learning results with γ = {4, 16, 32} on BoolQ, CB, and SciTail datasets. FT: Fine-tuning, AD: Adapter, PT: Prompt tuning, ST: SPoT, HF: HyperFormer, ATP: ATTEMPT. ACCEPT significantly outperforms other methods on BoolQ and offers comparable performance on CB and SciTail with fewer parameters.

LC	PS	PP	AP	SuperGLUE
✗	✗	✓	✗	60.0
✓	✗	✓	✗	75.5
✓	✓	✓	✗	**76.3**
✗	✗	✓	✓	76.5
✓	✗	✓	✓	77.7
✓	✓	✓	✓	**78.3**

Table 4: Effectiveness of learnable codebook and subdivision. Our designs of shared learnable codebook (LC) and prompt embedding subdivision (PS) allow a performance gain with the same scale of parameters (76.8k) as other approaches. PP and AP denote the prepended and added prompt tunings, respectively.

Soft-weighted Codebook Prepended Prompt (*SCPP*)						
(t, r)	(32, 20)	(64, 30)	(128, 40)	(256, 48)	(384, 51)	(768, 55)
#Para.	74,880	75,360	75,840	76,224	76,008	76,260
Acc.	**82.73**	79.14	79.14	77.70	**82.73**	81.29
Soft-weighted Codebook Added Prompt (*SCAP*)						
(t, r)	(32, 4)	(64, 8)	(128, 13)	(256, 20)	(384, 24)	(768, 30)
#Para.	73,728	76,800	76,032	76,800	76,800	76,800
Acc.	77.70	76.98	79.86	81.29	**82.73**	78.42

Table 5: Performance on RTE dataset with dividing *SCPP* and *SCAP* into different granularities. For T5-base, $t = 768$ means the prompt is NOT divided. Configs surpass non-division settings are highlighted in gray.

SCPP	*SCAP*	GLUE	SuperGLUE	MRQA
✗	✗	85.9	76.5	73.0
✓	✗	87.1	77.8	73.5
✗	✓	87.0	77.5	73.3
✓	✓	**87.9**	**78.3**	**73.6**

Table 6: Effectiveness of Soft-weighted Codebook Prepended and Added prompts. The optimal performance is achieved by both *SCPP* and *SCAP* combined.

Method	GLUE
PT	85.6
DePT	86.4
TPT	88.4
ACCEPT (Ours)	**88.5**

Table 7: Performance on GLUE with T5-3B. We outperform all PT, DePT and achieve a 0.1 improvement over TPT with less parameters.

Method	RTE
PT	88.49
DePT	89.92
ACCEPT (Ours)	**91.37**

Table 8: Performance of Flan-T5-11B model on RTE dataset. Our method outperforms both PT and DePT.

rameters are used compared to PT with $m = 100$. Note that with T5-base model, $t = 768$ means no division on the embedding dimension. Tab. 5 shows that with an appropriate division, there is a performance gain compared to treating the embedding as a whole. *SCPP* achieves an optimal performance with $t = 32$, chosen for its fewer parameters than $t = 384$. For *SCAP*, optimal performance is achieved with $t = 384$. More complete experiments are in Appendix B.1. Note that the optimal parameters ($t = 32, K = 24, r = 20$ for *SCPP* and $t = 384, K = 2, r = 24$ for *SCAP*) chosen from this small task (RTE) are then applied to ALL the datasets when using our approach with the T5-base model in the experiments.

Ablation on *SCPP* and *SCAP*. We train *SCPP* and *SCAP* individually, initializing them with a random Gaussian distribution. Tab. 6 shows that when using *SCPP* individually, the average performance on GLUE, SueprGLUE and MRQA improves 1.2%, 1.3% and 0.5%, respectively. Similarly, there are 1.1%, 1.0% and 0.3% performance gain when using *SCAP* individually. We find that the performance gain is relatively small on QA tasks. Improvement in the generation of longer sentences with ACCEPT is left as a future work. With both *SCPP* and *SCAP*, our approach achieves the best performances with 2.0%, 1.8% and 0.6% gain on each of the three benchmark, indicating the importance of both our designs. More detailed results on each dataset are in Appendix B.2.

Model Scaling. We explore the effect of different

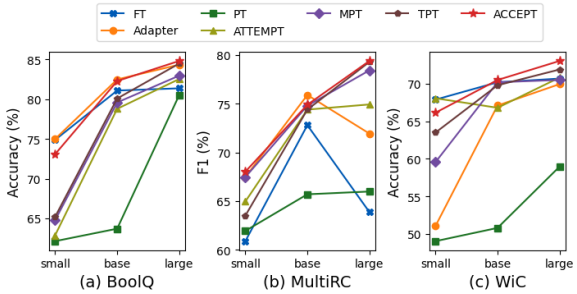


Figure 3: Performance on BoolQ, MultiRC and Wic datasets with different model sizes (T5-small, T5-base and T5-large). Our method shows improved performance as the model size increases and reaches SOTA on larger model, showcasing the potentional of ACCEPT.

model sizes (T5-small, T5-base and T5-large) with our method on BoolQ, MultiRC and WiC datasets in Fig. 3. Our method demonstrates increased performance improvement with larger language model backbones, highlighting ACCEPT's adaptability with bigger models. We also provide the results of fully fine-tuning (FT), Adapter, Prompt Tuning (PT), MPT, and TPT for comparison. ACCEPT demonstrates competitive performance across all model scales. Notably, the tunable parameters of our approach are much fewer than those in FT, Adapter, and ATTEMPT. Despite this, we achieve state-of-the-art performance on all three datasets with T5-large (770M), which is a highly encouraging result given the reduced parameter count.

To further study the capabilities and possibilities of ACCEPT on large language models, We conduct the experiments with billion-parameter models including T5-3B and Flan-T5-11B. Tab. 7 shows that ACCEPT achieves the state-of-the-art average accuracy on GLUE benchmark with T5-3B. ACCEPT surpasses the vanilla PT and other prompt tuning methods including DePT and TPT. Notably, we achieve a 0.1% improvement with much fewer parameters than TPT, which is an impressive result. Flan-T5 is an enhanced version of T5 model by fine-tuning T5 on 1,800 downstream tasks. We further select the 11 billion-parameter version and investigate the effectiveness of ACCEPT on large language models. Due to the huge computation resource required, we select the RTE dataset for evaluation.Tab. 8 shows that ACCEPT outperforms both PT and DePT on the RTE dataset. This indicates the potential and capability of ACCEPT incorporating large-scale models.

Prompt Initialization. We further analyze how initialization affects the performance. We conduct three initialization settings, (1) Random initialization: Both the codebooks and weights are initialized with a random Gaussian Distribution; (2) Intermediate task initialization: SPoT (Vu et al., 2022) has shown that initializing prompts with the pretrained weights from the tasks of a similar nature can benefit the training of the target task. (3) Target task initialization: By first pretraining the codebooks and the weights of *SCPP* and *SCAP* respectively on the target task, both of them are then served as the initialization of ACCEPT. Tab. 9 shows that our method achieves better performances than PT and DePT with all three strategies, revealing the robustness and effectiveness of ACCEPT. Moreover, intermediate task initialization strategy yields the best performances. We conjecture that the pretrained codebooks and weights from intermediate task of a similar nature helps the target task transfer more easily, providing additional knowledge and surpasses the performances of random or target task initialization. Detailed results are provided in Appendix B.3.

Method	GLUE	SuperGLUE
PT†	84.8	60.0
DePT	85.9	76.5
Init. method of ACCEPT		
Random	87.1	76.5
Intermediate task	**87.5**	**77.6**
Target task	87.1	77.2

Table 9: Performance of ACCEPT on GLUE and SuperGLUE with different prompt initialization. All three strategies outperforms PT and DePT, showing our method's robustness.

5 Conclusion

In this paper, we present ACCEPT, a novel prompt tuning method based on product quantization. As compared with other PT methods, the proposed method allows versatile and efficient prompt learning by subdividing prompt embeddings and computing each subprompt with the linear combination of learnable codewords and weights. Extensive experiments demonstrate that ACCEPT achieves outstanding performance across various NLP tasks. Furthermore, we also show the proposed approach is capable of being effectively adapted to billion-parameter models and achieves decent results.

While we currently use all codewords for linear combination, we aim to explore sparse representations in the future work. Besides, we plan to extend our research scope by applying ACCEPT to a wider range of tasks with a more diverse set of LLMs.

6 Limitations

While our extensive experiments across 17 datasets highlight the effectiveness of ACCEPT, it's important to acknowledge some additional considerations. Our method introduces some extra hyperparameters, such as determining the optimal sub-dimension t, which requires some extra computational efforts. Moreover, ACCEPT involves managing two distinct learning rates for *SCPP* and *SCAP*. Additionally, due to the significant resource requirements of the models with tens of billions of parameters, our experiments were conducted on a limited number of datasets. Future work will aim to explore ACCEPT on a broader range of datasets and larger models to further validate its efficacy.

References

Stefanos Angelidis, Reinald Kim Amplayo, Yoshihiko Suhara, Xiaolan Wang, and Mirella Lapata. 2021. Extractive opinion summarization in quantized transformer spaces. *Transactions of the Association for Computational Linguistics*, 9:277–293.

Akari Asai, Moh ammadreza Salehi, Matthew E Peters, and Hannaneh Hajishirzi. 2022. Attempt: Parameter-efficient multi-task tuning via attentional mixtures of soft prompts. In *EMNLP*.

Rishabh Bhardwaj, Amrita Saha, Steven CH Hoi, and Soujanya Poria. 2022. Vector-quantized input-contextualized soft prompts for natural language understanding. In *EMNLP*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *NeurIPS*.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *NAACL*.

Marie-Catherine De Marneffe, Mandy Simons, and Judith Tonhauser. 2019. The commitmentbank: Investigating projection in naturally occurring discourse. In *proceedings of Sinn und Bedeutung*, pages 107–124.

Dorottya Demszky, Kelvin Guu, and Percy Liang. 2018. Transforming question answering datasets into natural language inference datasets. *arXiv preprint arXiv:1809.02922*.

Bill Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Third international workshop on paraphrasing (IWP2005)*.

Matthew Dunn, Levent Sagun, Mike Higgins, V Ugur Guney, Volkan Cirik, and Kyunghyun Cho. 2017. Searchqa: A new q&a dataset augmented with context from a search engine. *arXiv preprint arXiv:1704.05179*.

Adam Fisch, Alon Talmor, Robin Jia, Minjoon Seo, Eunsol Choi, and Danqi Chen. 2019. Mrqa 2019 shared task: Evaluating generalization in reading comprehension. In *Proceedings of the 2nd Workshop on Machine Reading for Question Answering*.

Tianyu Gao, Adam Fisch, and Danqi Chen. 2021. Making pre-trained language models better few-shot learners. *IJCNLP*.

Danilo Giampiccolo, Bernardo Magnini, Ido Dagan, and William B Dolan. 2007. The third pascal recognizing textual entailment challenge. In *Proceedings of the ACL-PASCAL workshop on textual entailment and paraphrasing*, pages 1–9.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *ICML*, pages 2790–2799. PMLR.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. In *ICLR*.

Hamish Ivison and Matthew E Peters. 2022. Hyperdecoders: Instance-specific decoders for multi-task nlp. In *EMNLP*.

Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence*, 33(1):117–128.

Daniel Khashabi, Snigdha Chaturvedi, Michael Roth, Shyam Upadhyay, and Dan Roth. 2018. Looking beyond the surface: A challenge set for reading comprehension over multiple sentences. In *NAACL*, pages 252–262.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *EMNLP*.

Hector Levesque, Ernest Davis, and Leora Morgenstern. 2012. The winograd schema challenge. In *Thirteenth international conference on the principles of knowledge representation and reasoning*.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. In *IJCNLP*.

Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *ACL*.

Rabeeh Karimi Mahabadi, Sebastian Ruder, Mostafa Dehghani, and James Henderson. 2021. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. In *ACL*.

Giangiacomo Mercatali and André Freitas. 2021. Disentangling generative factors in natural language with discrete variational autoencoders. In *EMNLP*.

Mohammad Taher Pilehvar and Jose Camacho-Collados. 2018. Wic: the word-in-context dataset for evaluating context-sensitive meaning representations. In *NAACL*.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *EMNLP*.

Aurko Roy and David Grangier. 2019. Unsupervised paraphrasing without translation. In *ACL*.

Aurko Roy, Ashish Vaswani, Arvind Neelakantan, and Niki Parmar. 2018. Theory and experiments on vector quantized autoencoders. *arXiv preprint arXiv:1805.11063*.

Andreas Rücklé, Gregor Geigle, Max Glockner, Tilman Beck, Jonas Pfeiffer, Nils Reimers, and Iryna Gurevych. 2021. Adapterdrop: On the efficiency of adapters in transformers. *EMNLP*.

Timo Schick and Hinrich Schütze. 2021. It's not just size that matters: Small language models are also few-shot learners. In *NAACL*.

Zhengxiang Shi and Aldo Lipani. 2024. Dept: Decomposed prompt tuning for parameter-efficient fine-tuning. In *ICLR*.

Taylor Shin, Yasaman Razeghi, Robert L Logan IV, Eric Wallace, and Sameer Singh. 2020. Autoprompt: Eliciting knowledge from language models with automatically generated prompts. *EMNLP*.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *EMNLP*, pages 1631–1642.

Yusheng Su, Xiaozhi Wang, Yujia Qin, Chi-Min Chan, Yankai Lin, Huadong Wang, Kaiyue Wen, Zhiyuan Liu, Peng Li, Juanzi Li, et al. 2021. On transferability of prompt tuning for natural language processing. In *NAACL*.

Yi-Lin Sung, Jaemin Cho, and Mohit Bansal. 2022. Lst: Ladder side-tuning for parameter and memory efficient transfer learning. In *NeurIPS*, volume 35, pages 12991–13005.

Adam Trischler, Tong Wang, Xingdi Yuan, Justin Harris, Alessandro Sordoni, Philip Bachman, and Kaheer Suleman. 2016. Newsqa: A machine comprehension dataset. In *ACL*.

Aaron Van Den Oord, Oriol Vinyals, et al. 2017. Neural discrete representation learning. *NeurIPS*, 30.

Tu Vu, Brian Lester, Noah Constant, Rami Al-Rfou, and Daniel Cer. 2022. Spot: Better frozen model adaptation through soft prompt transfer. In *ACL*.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2019. Superglue: A stickier benchmark for general-purpose language understanding systems. *NeurIPS*, 32.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *ICLR*.

Sinong Wang, Han Fang, Madian Khabsa, Hanzi Mao, and Hao Ma. 2021. Entailment as few-shot learner. *arXiv preprint arXiv:2104.14690*.

Zhen Wang, Rameswar Panda, Leonid Karlinsky, Rogerio Feris, Huan Sun, and Yoon Kim. 2023. Multitask prompt tuning enables parameter-efficient transfer learning. In *ICLR*.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2019. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics*, 7:625–641.

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*.

Jiaxiang Wu, Cong Leng, Yuhang Wang, Qinghao Hu, and Jian Cheng. 2016. Quantized convolutional neural networks for mobile devices. In *CVPR*, pages 4820–4828.

Muling Wu, Wenhao Liu, Jianhan Xu, Changze Lv, Zixuan Ling, Tianlong Li, Longtao Huang, Xiaoqing Zheng, and Xuan-Jing Huang. 2023. Parameter efficient multi-task fine-tuning by learning to transfer token-wise prompts. In *EMNLP*, pages 8734–8746.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *EMNLP*.

Tan Yu, Junsong Yuan, Chen Fang, and Hailin Jin. 2018. Product quantization network for fast image retrieval. In *ECCV*, pages 186–201.

Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. 2022. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *ACL*.

Sheng Zhang, Xiaodong Liu, Jingjing Liu, Jianfeng Gao, Kevin Duh, and Benjamin Van Durme. 2018. Record: Bridging the gap between human and machine commonsense reading comprehension. *arXiv preprint arXiv:1810.12885*.

Appendix

A Experimental Setting

We use PyTorch[1], huggingface transformers[2] and huggingface PEFT[3] to implement our work. GLUE[4], SuperGLUE[5] and MRQA 2019 Shared Task[6] are downloaded from huggingface dataset. We use the original T5 checkpoint rather than the LM-adapted 1.1 version (Lester et al., 2021). We modified codes based on DePT's github repository[7]. We mainly cite the experiment results from (Wu et al., 2023) and (Shi and Lipani, 2024). We typically use $m = 60$ for the length of *SCPP*, and set the maximum sequence length l to 256, which also corresponds to the length of *SCAP* (except using 348 for MultiRC following Shi and Lipani (2024)). We partition *SCPP* and *SCAP* into $K = 24$ and $K = 2$ subsections, respectively. The associated r is calculated by the equation $rd + rmK \leq md$ for each model with dimension d. We primarily use a grid search to determine the learning rates (lr) for both the codebook and weights in *SCPP* and *SCAP*. For *SCPP*, the lr searched are {3e-1, 4e-1, 5e-1}, while for *SCAP*, we searched {1e-4, 5e-4, 1e-3, 5e-3}. Additionally, we observe that a larger lr is more suitable for *SCAP* on the MRQA 2019 Shared Task. Therefore, we extend our search to include higher values $\{1, 5, 10\}$ for *SCAP*. Note that for the experiments that train *SCPP* or *SCAP* alone, the backbone follows DePT (length of prompt is 60 and rank of LoRA matrices equals to 30). We train $30k$ steps for small datasets with less than $10k$ samples, and $300k$ steps for large datasets more than $10k$ samples, following Vu et al. (2022). We perform evaluations every 1,000 steps and save the best checkpoint based on performance on the evaluation dataset. The results on the test dataset are then reported using these best checkpoints. We choose a batch size of 16 for T5-small, T5-base and T5-large models, 2 for T5-3B and 1 for Flan-T5-11B due to the GPU memory limitation. The warmup step and weight decay are $1,800$ and 0.01, respectively. Experiments are conducted on a single Nvidia 3090 GPU with 24 GB memory or multiple Nvidia V100 GPUs with 32 GB memory.

[1] https://pytorch.org/
[2] https://github.com/huggingface/transformers
[3] https://github.com/huggingface/peft
[4] https://huggingface.co/datasets/glue
[5] https://huggingface.co/datasets/super_glue
[6] https://huggingface.co/lucadiliello
[7] https://github.com/ZhengxiangShi/DePT

B More Details of Experiments

In this section, we present more comprehensive experiments.

B.1 Details of Different Granularity of Subdivision.

We have shown the performance of different subdimension (t) and codebook size (r) pairs in Tab. 5. We present more results in Tab. 11 by selecting total 8 factors of the model dimension ($d = 768$ for T5-base) and conduct the experiments for each setting on RTE dataset. Tab. 11 shows that with an appropriate division, multiple configurations surpass the performance of not dividing prompts ($t = 768$), which demonstrate the effectiveness of PQ. We select $t = 32$, $K = 24$, $r = 20$) for *SCPP* and $t = 384$, $K = 2$, $r = 24$) for *SCAP* as the final decision considering both the performance and parameter efficiency, applying to all datasets.

B.2 Details of Ablation on *SCPP* and *SCAP*.

In the main paper, we provide the average performances on GLUE/SuperGLUE benchmarks and MRQA 2019 Shared Task in Tab. 6. Here we provide the performance on each dataset in Tab. 12 and Tab. 14. The results of most datasets show improvements when using either *SCPP* or *SCAP* individually, and are the best performances when both are applied simultaneously, further validating the effectiveness of ACCEPT.

B.3 Details of Prompt Initialization

Tab. 13 and Tab. 15 present the results for each dataset using three initialization strategies. The majority of performances improved with either intermediate task initialization or target task initialization, demonstrating the effectiveness of prelearning knowledge before transferring it to the target tasks, aligning with research (Vu et al., 2022).

C More Studies on Prompt Length.

We evaluate the impact of different prompt lengths (m) on model performance and training time, as shown in Fig. 4. The experiments are conducted on the MRPC and STS-B datasets with m values of $\{0, 20, 40, 60, 80, 100\}$, while maintaining the same level of training parameters across all settings. The results indicate that as m increases, the training time also rises. Notably, our approach achieves peak accuracy in both datasets with $m = 60$, mak-

Dataset Name	Benchmark	#Train	#Valid	#Test	Task Type	Metric
MNLI	GLUE	392,702	9,832	9,815	Natural Language Inference (NLI)	accuracy
QQP	GLUE	362,846	1,000	40,431	Paraphrase Detection	accuracy/F1
QNLI	GLUE	103,743	1,000	5,463	NLI	accuracy
SST-2	GLUE	66,349	1,000	872	Sentiment Analysis	accuracy
STS-B	GLUE	5,749	750	750	Sentence Similarity	Pearson/Spearman corr.
MRPC	GLUE	3,668	204	204	Paraphrase Detection	accuracy/F1
RTE	GLUE	2,490	138	139	NLI	accuracy
CoLA	GLUE	8,551	521	522	Acceptability	Matthews corr.
MultiRC	SuperGLUE	27,243	2,424	2,424	Question Answering (QA)	F1/EM
BoolQ	SuperGLUE	9,427	1,635	1,635	Boolean QA	accuracy
WiC	SuperGLUE	5,428	319	319	Word Sense Disambiguation	accuracy
WSC	SuperGLUE	554	52	52	Commonsense Reasoning	accuracy
CB	SuperGLUE	250	28	28	NLI	accuracy
ReCoRD	SuperGLUE	137,484	1,370	15,176	Commonsense Reasoning	F1/EM
NaturalQuestions	MRQA 2019	103,071	1,000	12,836	Extractive QA	F1/EM
HotpotQA	MRQA 2019	71,928	1,000	5,901	Extractive QA	F1/EM
SearchQA	MRQA 2019	116,384	1,000	16,980	Extractive QA	F1/EM
NewsQA	MRQA 2019	73,160	1,000	4,212	Extractive QA	F1/EM
SQuAD	MRQA 2019	86,599	1,000	10,570	Extractive QA	F1/EM

Table 10: Detailed information of all datasets used in our experiments. For datasets that originally use two metrics, we designate the underlined metric as our primary evaluation measure following prior works (Asai et al., 2022; Shi and Lipani, 2024).

Soft-weighted Codebook Prepended Prompt (*SCPP*)									
(t, r)	(16, 12)	(32, 20)	(64, 30)	(96, 36)	(128, 40)	(192, 45)	(256, 48)	(384, 51)	(768, 55)
#Para.	74496	74880	75360	75648	75840	76080	76224	76008	76260
Acc.	77.70	**82.73**	79.14	**82.73**	79.14	81.29	77.70	**82.73**	81.29
Soft-weighted Codebook Added Prompt (*SCAP*)									
(t, r)	(16, 2)	(32, 4)	(64, 8)	(96, 10)	(128, 13)	(192, 17)	(256, 20)	(384, 24)	(768, 30)
#Para.	72192	73728	76800	74240	76032	76544	76800	76800	76800
Acc.	78.42	77.70	76.98	76.98	79.86	78.42	81.29	**82.73**	78.42

Table 11: Performance on RTE dataset with dividing the *SCPP* and *SCAP* into different granularities. Note that for T5-base, $t = 768$ means the prompt is NOT divided.

SCPP	*SCAP*	**GLUE**									**SuperGLUE**					
		MNLI	QQP	QNLI	SST-2	STS-B	MRPC	RTE	CoLA	Avg.	Multi	Bool	WiC	WSC	CB	Avg.
✗	✗	85.0	**90.4**	**93.2**	94.2	90.8	90.7	79.1	63.8	85.9	74.3	79.3	68.7	**67.3**	92.9	76.5
✓	✗	85.9	90.3	**93.2**	94.3	91.0	91.7	82.7	67.5	87.1	74.3	80.9	70.2	**67.3**	**96.4**	77.8
✗	✓	**86.0**	**90.4**	**93.2**	94.3	**91.1**	90.7	82.7	66.8	87.0	**75.4**	81.2	67.4	**67.3**	**96.4**	77.5
✓	✓	85.9	**90.4**	93.1	**94.5**	91.0	**93.1**	**86.3**	**68.8**	**87.9**	74.9	**82.3**	**70.5**	**67.3**	**96.4**	**78.3**

Table 12: Ablation study of *SCPP* and *SCAP* on GLUE and SuperGLUE benckmarks. We provide the performance of each dataset.

Init. Method	**GLUE**									**SuperGLUE**					
	MNLI	QQP	QNLI	SST-2	STS-B	MRPC	RTE	CoLA	Avg.	Multi	Bool	WiC	WSC	CB	Avg.
Random	85.7	90.2	93.0	94.3	**91.0**	**93.1**	84.2	65.2	87.1	74.5	81.0	**70.5**	**67.3**	92.9	77.2
Intermediate task	**85.9**	90.2	**93.3**	94.2	**91.0**	92.7	**86.3**	66.4	**87.5**	73.5	**82.3**	68.7	**67.3**	**96.4**	**77.6**
Target task	**85.9**	**90.4**	93.1	**94.5**	**91.0**	91.7	81.3	**68.8**	87.1	**74.9**	81.8	69.0	**67.3**	92.9	77.2

Table 13: Performance on GLUE and SuperGLUE with different prompt initialization.

ing it our optimal choice for the prompt length setting.

D Task and Dataset Details

We list the detailed information, including numbers of training, evaluation and testing samples, task

SCPP	*SCAP*	MRQA				
		NQ	HP	SQA	News	Avg.
✗	✗	$73.2_{0.1}$	$76.8_{0.3}$	$77.6_{0.2}$	$64.4_{0.1}$	73.0
✓	✗	$73.8_{0.05}$	$76.9_{0.01}$	$78.8_{0.2}$	$64.7_{0.1}$	73.5
✗	✓	$73.4_{0.2}$	$76.8_{0.5}$	$78.5_{0.2}$	$64.3_{0.1}$	73.3
✓	✓	$73.6_{0.05}$	$77.1_{0.1}$	$78.9_{0.01}$	$64.6_{0.06}$	**73.6**

Table 14: Ablation study of *SCPP* and *SCAP* on MRQA 2019 Shared Task. We report the average F1 and standard deviation of three experiments with different seeds.

Init. Method	MRQA				
	NQ	HP	SQA	News	Avg.
Random	73.47	76.74	78.59	**64.63**	73.36
Intermediate task	72.71	76.98	78.47	64.44	73.15
Target task	**73.61**	**77.10**	**78.91**	64.62	**73.55**

Table 15: Performance on MRQA 2019 Shared Task with different prompt initialization.

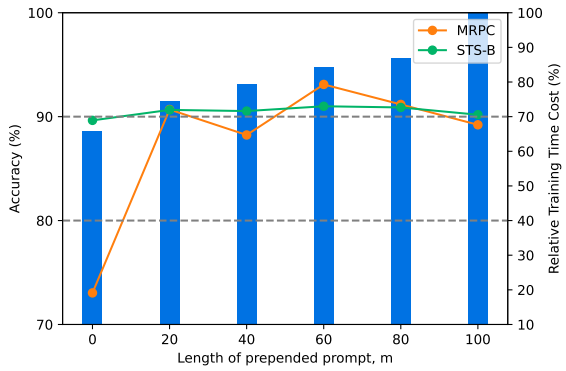


Figure 4: Performance on the MRPC and STS-B datasets and their relative training time (normalized to the one with $m = 100$) for various prompt lengths $m = \{20, 40, 60, 80, 100\}$. Both datasets show the best performance at $m = 60$.

types and evaluation metrics of each dataset which has been used in our experiments in Tab. 10. We utilize a diverse range of datasets covering various NLU tasks, including Natural Language Inference (NLI), Paraphrase Detection, and Sentiment Analysis. Additionally, we explore different types of Question Answering (QA) tasks, such as extractive and boolean QA. The effectiveness and generalizability of ACCEPT are demonstrated across these tasks in Tab. 1 and Tab. 2.